IBM BOB HACKATHON · THEME 2:
MODERNIZE WHAT MATTERS

FIVE YEARS OF COPY AND PASTE, CLEANED UP BY IBM BOB

Academy Sim Racing, Tampa FL



MALIKAI MADRAY

GITHUB.COM/ACADEMYSIM/SEO-BUILD-BOB

01 TEAM AND PROJECT

A TEAM OF ONE, AND FIVE LIVE WEBSITES

I'm Malikai, owner of Academy Sim Racing, a sim racing lounge in Tampa. I'm not a developer. About five years ago I built our five websites from YouTube guides: lounge, events, builds, driver school and health.

Most of our customers now find us through those sites, so they can't go down while being fixed.

The project: modernize the Python code that builds all five sites, using IBM Bob, without changing anything a visitor sees.



02 THE PROBLEM

EVERY FIX HAD TO BE MADE FIVE TIMES

- Five audit scripts, 129 to 194 lines each, each with its own copy of the same checks
- Four near-identical scripts for linking the sites together
- Tracking tag, site addresses and phone number typed out in many files
- 13 hand-written test cases and no test runner
- 136 shipped files that nothing checked at all
- One wrong byte can break a booking form, a tracking tag or a search ranking

BUILD THE SAFETY NET FIRST

Every file the sites ship got a SHA-256 fingerprint from the pre-Bob code: 263 of them. 127 were recorded before Bob started; Bob added the other 136 from a pre-Bob build.

Every step after that had to end with the fingerprints matching and the tests passing. Bob worked from a written plan, one phase per commit, with deploy credentials switched off.

The demo proves the net works: one added character in the lounge homepage file is caught, undone, and everything matches again.

```
Edited lounge/upload/index.html: added one '!'
> python shared/tools/verify_baseline.py
OK  127 files match BASELINE-site-hashes.txt
CHANGED : lounge/upload/index.html

1/263 files FAILED
exit 1
Change undone.
OK  127 files match BASELINE-site-hashes.txt
OK  136 files match BASELINE-upload-hashes.txt
exit 0
```

BOB DID THE WORK, PHASE BY PHASE

Plan	Oct 5	Read the legacy code and wrote `MODERNIZATION-PLAN.md`: phases P0 to P3
P0-P1	Oct 5	Fingerprint check, test scaffold, `constants.py`, fingerprints extended to all 136 shipped files
P2	Oct 5	Tests that lock in old behaviour first, then one shared `audit_lib.py`; all five audits moved onto it, one commit each
P3	Oct 5	Tests for page generation, form handling, breadcrumbs; dry-run mode for the full build
CI	Oct 14	GitHub runs every check on every push (one CI step later fixed by hand); read-only audit of the live sites (115 pages, 0 failures)
More	Oct 15	Offline tests for the network tools; shared `paths.py` for 9 scripts; architecture diagram

Every commit is listed in the repo (`BOB-COMMITS.txt` and the git history) and on the demo page.

05 RESULTS

CLEANER CODE, SAME WEBSITES

263

shipped files still byte-identical

202

automated tests, up from 13

5→1

copies of the audit checks

115

live pages audited, 0 failures

A fresh clone rebuilds all five sites and passes every check on GitHub. The five site audits print the same results they did before the refactor.

WHAT WE LEARNED, WHAT COMES NEXT

LEARNED

- Write the safety check before the refactor. It's what made it safe to let an AI agent touch live code.
- Lock in old behaviour with tests before moving it. "It still runs" isn't proof.
- Start Bob in the right folder. Twice it ran in the wrong one and built from scratch instead.
- Check the write-up like the code. One summary called a partial refactor done; we corrected it.

NEXT

- Move the remaining 42 older scripts onto `paths.py`
- Bring the deploy tools under tests against a staging site
- Make the fingerprint check a required step before every deploy
- Run the live-site audit on a schedule

LINKS

SEE IT FOR YOURSELF

Demo video	youtu.be/fh56qSFPcsE
Demo page	seo-bob-build.netlify.app
Code	github.com/academysim/seo-build-bob

```
git clone https://github.com/academysim/seo-build-bob
cd seo-build-bob
python shared/tools/build_all.py
python shared/tools/verify_baseline.py
python -m unittest discover -s shared/tests
```